Chapter 15 worked solutions

Delta Rules and Hybrid Stacks

This guide works through Exercises 1–4 and 9–10. It follows prediction, correction, and readout in a matrix state, counts the history that different layers keep, and examines what happens when two associations share a key direction. The companion notebook covers Exercises 5–8 and is also available as a PDF.

1. Correct one column

A matrix state stores a value vector along each key coordinate. Here both the key and the query are the second coordinate vector, $\mathbf{k} = \mathbf{q} = (0, 1)^\top$. Multiplying the state by this vector reads column two. We first decay the state by $\alpha = 1/2$ and use the key to read the resulting prediction.

$$\tilde{S} = \frac{1}{2}\begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 1/2 & 1 \\ 0 & 3/2 \end{bmatrix}, \qquad \widehat{\mathbf{v}} = \tilde{S}\mathbf{k} = \begin{bmatrix} 1 \\ 3/2 \end{bmatrix}.$$

The new value is $(4, -1)^\top$. Subtract the prediction from this value to obtain the error.

$$\mathbf{e} = \mathbf{v} - \widehat{\mathbf{v}} = \begin{bmatrix} 4 - 1 \\ -1 - 3/2 \end{bmatrix} = \begin{bmatrix} 3 \\ -5/2 \end{bmatrix}.$$

The outer product $\mathbf{e}\mathbf{k}^\top$ puts a zero in the first column because the first key coordinate is zero. It puts the entire error in the second column because the second coordinate is one. The correction coefficient $\beta = 1/4$ then scales that error before it is added.

$$\beta\mathbf{e}\mathbf{k}^\top = \begin{bmatrix} 0 & 3/4 \\ 0 & -5/8 \end{bmatrix},$$

$$S_{new} = \begin{bmatrix} 1/2 & 1 \\ 0 & 3/2 \end{bmatrix} + \begin{bmatrix} 0 & 3/4 \\ 0 & -5/8 \end{bmatrix} = \begin{bmatrix} 1/2 & 7/4 \\ 0 & 7/8 \end{bmatrix}.$$

The query reads the **updated** second column, so the output is $(7/4, 7/8)^\top$, or $(1.75, 0.875)^\top$. It lies one quarter of the way from the decayed prediction to the new value. The first column was originally $(1, 0)^\top$ and became $(1/2, 0)^\top$ through decay alone. The write affects only the addressed column in this example.

2. Trace a sign change

The key is the first coordinate vector $\mathbf{e}_1 = (1, 0)^\top$, and the value to write is zero. The update can therefore be read as a transition on the old state. For each choice of β, restart from the identity $S = I$.

$$\mathbf{k}\mathbf{k}^\top = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \qquad S_{new} = 0.6I(I - \beta\mathbf{k}\mathbf{k}^\top) = \begin{bmatrix} 0.6(1-\beta) & 0 \\ 0 & 0.6 \end{bmatrix}.$$

Read the two directions by multiplying by $\mathbf{e}_1$ and $\mathbf{e}_2 = (0, 1)^\top$. These reads select the first and second columns, respectively.

Correction β	New state S_{new}	Read at $\mathbf{e}_1$	Read at $\mathbf{e}_2$
0.5	$\mathrm{diag}\,(0.3, 0.6)$	$(0.3, 0)^\top$	$(0, 0.6)^\top$
1	$\mathrm{diag}\,(0, 0.6)$	$(0, 0)^\top$	$(0, 0.6)^\top$
1.5	$\mathrm{diag}\,(-0.3, 0.6)$	$(-0.3, 0)^\top$	$(0, 0.6)^\top$

At the key, the decayed prediction is $(0.6, 0)^\top$ in all three runs. Correcting it halfway toward zero leaves $(0.3, 0)^\top$. A full correction reaches zero. A correction of one and a half times the error passes through zero, leaving a value of equal magnitude to the first case but opposite sign. This is the change in the carry coefficient $0.6(1 - \beta)$. The second direction is orthogonal to the key, so its read changes only through the common decay of 0.6.

3. Read the Jamba schedule

The figure shows one block of eight layers. Its mixer row marks attention at position 5 and Mamba everywhere else. The separate feed-forward row marks MoE at positions 2, 4, 6, and 8, with a dense feed-forward network at the other positions. All four MoE positions therefore follow a recurrent mixer within their respective layers.

For the full stack, add 0, 8, 16, or 24 to each position in the block. A growing key–value cache stores an entry for each token at an attention layer, so find the cache layers by adding those offsets to position 5.

Block	Full-stack layer numbers	Layer with a growing KV cache	Recurrent mixer with MoE
1	1–8	5	2, 4, 6, 8
2	9–16	13	10, 12, 14, 16
3	17–24	21	18, 20, 22, 24
4	25–32	29	26, 28, 30, 32

The growing KV caches are at layers **5, 13, 21, and 29**. Each recurrent layer keeps a fixed-size state and a short convolution buffer. Four layers per block combine a recurrent mixer with an MoE feed-forward sublayer, for **16 layers** across four blocks.

Layer 14 is position $14 - 8 = 6$ in the second block. The token first passes through the **Mamba recurrent mixer**, then through an **MoE feed-forward sublayer**. Its feed-forward input is routed to **two of that sublayer's 16 experts**. The architecture fixes the mixer at this layer, while the router selects experts using the token representation produced by the mixer.

4. Count bytes across different stores

For one sequence, each token adds a key of width three and a value of width two to the full-attention cache. These are stored at two bytes per scalar, so the cache grows by $(3 + 2) \times 2 = 10$ bytes per token. The two recurrent layers carry 12 and 8 scalars. At four bytes each, their persistent stores take $12 \times 4 = 48$ bytes and $8 \times 4 = 32$ bytes, regardless of prefix length.

For a prefix of T tokens, the persistent history for one sequence is

$$10T + 48 + 32 = 10T + 80 \quad \text{bytes.}$$

Prefix length T	Attention KV cache	First recurrent layer	Second recurrent layer	Total
7	70 bytes	48 bytes	32 bytes	**150 bytes**
14	140 bytes	48 bytes	32 bytes	**220 bytes**

Doubling the prefix doubles the cache contribution and adds 70 bytes to the total. The recurrent contribution stays at 80 bytes. This count includes the numeric payloads of the persistent stores. Model weights, temporary arrays, and allocation overhead belong to separate memory measurements.

9. Write to overlapping directions (optional)

There is one value coordinate, so the state is a row with two entries, initially $S_0 = (0, 0)$. A read at a column vector $\mathbf{q}$ returns the scalar $S\mathbf{q}$. With $\alpha = \beta = 1$, the update adds the full prediction error along the current unit key.

$$S_{new} = S + (v - S\mathbf{k})\mathbf{k}^\top.$$

In both runs, the first key is $\mathbf{k}_1 = (1, 0)^\top$ and the new value is 2. The zero state predicts 0, so the first write adds $2(1, 0)$ and leaves $S_1 = (2, 0)$. The read at the first key immediately after this write is 2.

With the **overlapping** second key $\mathbf{k}_2 = (1/\sqrt{2}\ , 1/\sqrt{2}\ )^\top$, the first write already contributes to the second key's prediction. It gives $S_1\mathbf{k}_2 = 2/\sqrt{2}\ = \sqrt{2}\ $. The second write corrects the remaining error, $5 - \sqrt{2}\ $.

$$\begin{aligned} S_2 \ &= (2, 0) + (5 - \sqrt{2}\ )\left(1/\sqrt{2}\ , 1/\sqrt{2}\ \right) \\ &= \left(1 + 5/\sqrt{2}\ ,\ 5/\sqrt{2}\ - 1\right). \end{aligned}$$

The read at the first key is now $1 + 5/\sqrt{2}\ \approx 4.536$. The read at the second key is the sum of the two state coordinates divided by $\sqrt{2}\ $, giving exactly 5.

For the **orthogonal** run, restart from zero and perform the same first write. This time the second key is $\mathbf{k}_2 = (0, 1)^\top$. The state $(2, 0)$ predicts 0 there, so the second write adds $5(0, 1)$ and leaves $S_2 = (2, 5)$. The final reads are 2 at the first key and 5 at the second.

Second key	First-key read before second write	First-key read after second write	Second-key read after second write
$(1/\sqrt{2}\ , 1/\sqrt{2}\ )^\top$	2	$1 + 5/\sqrt{2}\ \approx 4.536$	5
$(0, 1)^\top$	2	2	5

At any query $\mathbf{q}$, the correction from the second write changes the read by $(5 - S_1\mathbf{k}_2)\mathbf{k}_2^\top\mathbf{q}$. At the new key, its inner product with itself is one, so the full error is corrected in both runs. At the first key, the inner product with the overlapping key is $1/\sqrt{2}\ $. The correction therefore also raises the first read by $(5 - \sqrt{2}\ )/\sqrt{2}\ = 5/\sqrt{2}\ - 1$. In the orthogonal run the inner product is zero, so the first read remains 2.

10. Plan a controlled comparison (optional)

Put a table of delivery addresses at the start of a sequence. Later messages change which entry is active. To see the distinction between the three questions, take a small table with indices 0, 1, 2, 3 and addresses **Ash**, **Birch**, **Cedar**, and **Elm**. Set the initial active index to 2. Three later messages say *advance*, *advance*, and *retreat*. Use a circular list, so advancing from 3 returns to 0. The active indices after the three messages are 3, 0, and 3.

Question about the same history	Answer	Information needed
What address is at index 1?	Birch	Find the entry for a specified index in the early table.
What is the current active index?	3	Apply the moves to the initial index.
What is the address at the current active index?	Elm	Compute the current index, then retrieve its address.

Use tables with eight entries in the experiment. Draw eight distinct addresses from a shared pool of equally long address labels, then randomly assign them to the indices. Sample the initial index uniformly and generate a stream of advances and retreats. For each move $d_t \in \{-1, 1\}$, compute the reference index by

$$i_t = (i_{t-1} + d_t) \mod 8.$$

Report the initial index explicitly in the input. Choose the specified index for the first question independently of the current active index. The address mapping changes across sequences, so memorizing a fixed index-to-address rule cannot solve the tasks. With a uniform initial index, the final index remains uniform even when the final move is known. Include matched examples in which the final message is identical but changing the earlier moves or initial index changes the final index modulo eight. Include other examples in which the index trajectory is identical but the address at the queried index has been reassigned. These pairs help check whether the model uses the relevant part of the history.

Train on streams with 8 to 32 moves. Test on streams with 16 and 32 moves from held-out histories and on longer streams with 64 and 128 moves. Record their lengths in tokens as well. For each test length, keep the number of address entries fixed and vary the moves. Put the question after the last report. Use the same formatted input and answer vocabulary for each model. Assign whole families of tables and trajectories to splits before forming their questions. Check that neither a test address mapping nor a test pair of initial index and move sequence occurs in training, even when combined with a different partner. Reserve validation histories for configuration and checkpoint selection and keep the test histories separate.

Compare three causal stacks with the same number of layers, residual width, feed-forward layers, embedding, and output head. In one stack, use full attention in every mixer layer. In another, use recurrent mixers in every layer. In the hybrid, place the two mixer types in a fixed alternating order. Match the tokenizer, training and validation prompts, training token budget, optimizer, learning-rate schedule, and checkpoint selection rule. Start shared parameters from paired seeds and train each variant with several such seeds. Report any parameter-count or training-compute differences due to the mixers. The design matches the training exposure, so compute and parameter count remain quantities to report when interpreting the comparison.

Measure exact match for each question type and each test length. Require a single valid index or address under a common decoding rule. In addition to overall scores, report the fraction of histories on which all three answers are correct. For a more direct diagnostic, also ask the specified-index question with the true current index supplied. Getting the current index right and the current address wrong points to difficulty retrieving the address or using the index to select it. The separate questions help localize a failure, while the combined question tests whether the two operations work together.

For each length, measure **persistent history bytes per sequence** immediately after the last report and before appending the question. Use batch size one and float32 for the carried state and KV tensors in all three implementations. Count the live key and value arrays in attention layers and all state arrays that recurrent layers carry to the next step, including any carried convolution buffers. For example, a single matrix state of shape $[d_v, d_k]$ takes sd_vd_k bytes with s bytes per scalar. A full-attention layer with T stored positions, H_{KV} stored heads, and key and value widths d_k, d_v takes $sTH_{KV}(d_k + d_v)$ bytes for its KV payload. Sum the relevant terms across layers. A hybrid can have both a length-dependent term and a fixed state term. Report allocated capacity separately if there are unused reserved cache slots, and keep model parameters and temporary work out of the history count. Compare the three accuracy curves alongside their measured history sizes.